

Prefix Sum + HashMap Templates

Core idea: `subarray sum [l, r] = prefix[r+1] - prefix[l]` .
To find a subarray with a target property, rephrase as: "has a prefix with value X been seen before?"
Store that X in a HashMap for O(1) lookup.

Core Template

Variables: `map` = `prefixSum`→count or index seen so far · `sum` = running prefix sum · `lookup` = the complement prefix we search for · `i` = current scan position **Pseudocode:**

```
create empty map of prefix → (count or index)
seed map with prefix 0 (count=1 for counting, index=-1 for length)
sum = 0
for each index i:
    add nums[i] to sum
    lookup = transform(sum)          // sum-k, sum%k, etc.
    use map.get(lookup) to update result
    store sum (or sum%k) into map with count or index i

// a subarray is the difference of two prefixes; remember prefixes seen so far and look up the compleme
Map<Long, Integer> map = new HashMap<>();
map.put(0L, ???);    // seed: empty prefix (sum=0) seen before index 0
long sum = 0;
// result = 0 (count) or Integer.MAX_VALUE (length sentinel) depending on problem

for (int i = 0; i < nums.length; i++) {
    sum += nums[i];
    long lookup = transform(sum);      // what to look up (sum-k, sum%k, etc.)
    // use map.get(lookup) → update result
    map.put(storeKey(sum), storeValue(i)); // what to store (sum or sum%k → count or index)
}
```

What changes per problem:

Variable	Count problems (560, 974)	Length problems (325, 523)
Map value	count (how many times seen)	first index (earliest occurrence)
Seed value	map.put(0L, 1)	map.put(0L, -1)
On lookup hit	count += map.get(lookup)	maxLen = max(i - map.get(lookup))
Overwrite map?	Yes (merge count)	No (keep first index only)

Quick Reference Table

Read the two map-value modes first: **count** problems store `prefix sum` → how many times seen and answer "how many"; **length/index** problems store `prefix sum` → earliest index seen and answer "how long / where". The two modes also seed differently:

- Seed `(0, 1)` = the empty prefix has been **seen once** before we start — needed for *counting* (a prefix that itself equals the target counts as one valid subarray).
- Seed `(0, -1)` = the empty prefix lives at a **virtual index -1** — needed for *length* (length from start = `i - (-1) = i + 1`).

#	Name	Description	Intuition	Time	Space	Key Implementation
560	Subarray Sum Equals K	Count the number of subarrays whose sum equals k.	A subarray summing to k ends here for every earlier prefix equal to <code>sum - k</code> ; count those prefixes.	O(n)	O(n)	Standard
325	Maximum Size Subarray Sum Equals k	Find the length of the longest subarray summing to k.	Same complement as #560, but store the earliest index so <code>i - p</code> gives the longest subarray summing to k.	O(n)	O(n)	Standard
974	Subarray Sums Divisible by K	Count subarrays whose sum is divisible by k.	Two prefixes with the same remainder bracket a subarray divisible by k; count how many share each remainder.	O(n)	O(min(n, k))	Standard
523	Continuous Subarray Sum	Return true if any subarray of length ≥ 2 has sum divisible by k.	Same-remainder pair means a divisible subarray; store the earliest index so the gap <code>i - p ≥ 2</code> proves length ≥ 2 .	O(n)	O(min(n, k))	Standard
437	Path Sum III	Count the number of paths in a binary tree that sum to targetSum. Paths can start and end at any node but must go downward.	Apply the #560 prefix-count idea along each root-to-node path; undo the prefix on the way back up so branches stay independent.	O(n)	O(h) (recursion + path-prefix map, h = tree height)	Standard
1	Two Sum	Given an array and a target, return the indices of the two numbers that add up to target.	Same "have I seen the complement before?" idea as prefix-sum, but the key is the raw value, so one pass finds the pair.	O(n)	O(n)	look up complement, not prefix sum

#560 Subarray Sum Equals K

Description: Count the number of subarrays whose sum equals k.

Map: {prefix_sum → count}. Seed (0, 1) — the empty prefix.

Lookup: how many earlier prefixes equal `sum - k`? Each one forms a valid subarray ending here.

Intuition: A subarray summing to k ends here for every earlier prefix equal to `sum - k`; count those prefixes.

Variables: `map` = prefixSum → count of times seen · `sum` = running prefix sum · `count` = answer (subarrays summing to k) · `num` = current element **Pseudocode:**

```

create empty map of prefixSum → count
seed map with (0 → 1)           // empty prefix counts once
sum = 0, count = 0
for each num in nums:
    add num to sum
    count += how many earlier prefixes equal sum-k
    increment map count for sum
return count

```

```

class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Long, Integer> map = new HashMap<>();
        map.put(0L, 1);
        long sum = 0;
        int count = 0;
        for (int num : nums) {
            sum += num;
            count += map.getOrDefault(sum - k, 0);
            map.merge(sum, 1, Integer::sum);
        }
        return count;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#325 Maximum Size Subarray Sum Equals k

Description: Find the length of the longest subarray summing to k.

Map: {prefix_sum → first index}. Seed (0, -1) — empty prefix at virtual index -1.

Lookup: if $\text{sum} - k$ was seen at index p , length = $i - p$. Only keep first occurrence (maximizes length).

Intuition: Same complement as #560, but store the earliest index so $i - p$ gives the longest subarray summing to k.

Variables: map = prefixSum → first index seen · sum = running prefix sum · maxLen = longest subarray length found · i = current index **Pseudocode:**

```

create empty map of prefixSum → first index
seed map with (0 → -1)           // empty prefix at virtual index -1
sum = 0, maxLen = 0
for i from 0 to n-1:
    add nums[i] to sum
    if sum-k was seen before:
        maxLen = max(maxLen, i - index where sum-k was first seen)
    if sum not in map:
        store sum → i           // keep earliest only
return maxLen

```

```

class Solution {
    public int maxSubArrayLen(int[] nums, int k) {
        Map<Long, Integer> map = new HashMap<>();
        map.put(0L, -1);
        long sum = 0;
        int maxLen = 0;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            if (map.containsKey(sum - k))
                maxLen = Math.max(maxLen, i - map.get(sum - k));
            if (!map.containsKey(sum)) // only store first occurrence
                map.put(sum, i);
        }
        return maxLen;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#974 Subarray Sums Divisible by K

Description: Count subarrays whose sum is divisible by k .

Map: $\{\text{prefix_sum} \% k \rightarrow \text{count}\}$. Two subarrays share the same remainder $\rightarrow$ their difference is divisible by k .

Lookup: how many earlier prefixes have the same remainder?

Watch out: Java `%` can return negative — normalize with $((\text{sum} \% k) + k) \% k$.

Intuition: Two prefixes with the same remainder bracket a subarray divisible by k ; count how many share each remainder.

Variables: `map` = remainder(sum%k) $\rightarrow$ count of times seen · `sum` = running prefix sum mod k (normalized non-negative)
· `count` = answer · `num` = current element **Pseudocode:**

```
create empty map of remainder  $\rightarrow$  count
seed map with (0  $\rightarrow$  1)           // empty prefix has remainder 0
sum = 0, count = 0
for each num in nums:
    sum = (sum + num) mod k, normalized to be non-negative
    count += how many earlier prefixes share this remainder
    increment map count for sum
return count
```

```
class Solution {
    public int subarraysDivByK(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);
        int sum = 0, count = 0;
        for (int num : nums) {
            sum = ((sum + num) % k + k) % k;    // normalize negative remainders
            count += map.getOrDefault(sum, 0);
            map.merge(sum, 1, Integer::sum);
        }
        return count;
    }
}
```

Time $O(n)$ | **Space** $O(\min(n, k))$

#523 Continuous Subarray Sum

Description: Return true if any subarray of length ≥ 2 has sum divisible by k .

Map: $\{\text{prefix_sum} \% k \rightarrow \text{first index}\}$. Seed (0, -1).

Lookup: same remainder seen before $\rightarrow$ their difference is divisible by k . Check gap ≥ 2 .

Key difference from #974: store first index (not count), check gap, return on first hit.

Intuition: Same-remainder pair means a divisible subarray; store the earliest index so the gap $i - p \geq 2$ proves length ≥ 2 .

Variables: `map` = remainder(sum%k) $\rightarrow$ first index seen · `sum` = running prefix sum mod k (normalized non-negative) · `i` = current index **Pseudocode:**

```

create empty map of remainder → first index
seed map with (0 → -1)           // empty prefix at virtual index -1
sum = 0
for i from 0 to n-1:
    sum = (sum + nums[i]) mod k, normalized to be non-negative
    if remainder sum was seen before:
        if i - first index of sum >= 2: return true
    else:
        store sum → i           // keep earliest only
return false

```

```

class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, -1);
        int sum = 0;
        for (int i = 0; i < nums.length; i++) {
            sum = ((sum + nums[i]) % k + k) % k; // normalize negative remainders (matches #974)
            if (map.containsKey(sum)) {
                if (i - map.get(sum) >= 2) return true;
            } else {
                map.put(sum, i); // only store first occurrence
            }
        }
        return false;
    }
}

```

Time $O(n)$ | **Space** $O(\min(n, k))$

#437 Path Sum III

Description: Count the number of paths in a binary tree that sum to targetSum. Paths can start and end at any node but must go downward.

Map: {prefix_sum → count} along the current root-to-node path. Seed (0, 1).

Key difference: tree DFS — must **backtrack** (decrement count) when leaving a node, so sibling subtrees don't share state.

Intuition: Apply the #560 prefix-count idea along each root-to-node path; undo the prefix on the way back up so branches stay independent.

Variables: map = path prefixSum→count seen on current root-to-node path · sum = running prefix sum down the path · node = current tree node · count = paths ending at/below this node **Pseudocode:**

```

create map of prefixSum → count, seeded with (0 → 1)
dfs(root, sum=0):
    if node is null: return 0
    add node value to sum
    count = how many path prefixes equal sum-target
    increment map count for sum           // add this node's prefix
    count += dfs(left) + dfs(right)
    decrement map count for sum           // backtrack before returning
    return count

```

```

class Solution {
    public int pathSum(TreeNode root, int targetSum) {
        Map<Long, Integer> map = new HashMap<>();
        map.put(0L, 1);
        return dfs(root, 0L, targetSum, map);
    }

    private int dfs(TreeNode node, long sum, int target, Map<Long, Integer> map) {
        if (node == null) return 0;
        sum += node.val;
        int count = map.getDefault(sum - target, 0);
        map.merge(sum, 1, Integer::sum);
        count += dfs(node.left, sum, target, map);
        count += dfs(node.right, sum, target, map);
        map.merge(sum, -1, Integer::sum);    // backtrack - undo before returning
        return count;
    }
}

```

Time $O(n)$ | **Space** $O(h)$ (recursion + path-prefix map, h = tree height)

523 vs 974 — Side by Side

Both use `prefix sum % k` as the map key. The difference is one line:

	974 (count all)	523 (any length ≥ 2)
Map value:	count	first index
Seed:	<code>map.put(0, 1)</code>	<code>map.put(0, -1)</code>
On hit:	<code>count += map.get(sum)</code>	<code>if (i - map.get(sum) >= 2) return true</code>
Store:	<code>map.merge(sum, 1, +)</code>	<code>map.putIfAbsent(sum, i)</code>

560 vs 325 — Side by Side

Both use `prefix sum` as the map key, look up `sum - k`. The difference is one line:

	560 (count all)	325 (longest)
Map value:	count	first index
Seed:	<code>map.put(0L, 1)</code>	<code>map.put(0L, -1)</code>
On hit:	<code>count += map.get(...)</code>	<code>maxLen = max(i - map.get(...))</code>
Store:	<code>map.merge(sum, 1, +)</code>	<code>map.putIfAbsent(sum, i)</code>

When to Use `long` vs `int`

- Use `long` for the prefix sum when values can be large (e.g., `nums[i]` up to $10^4 \times n$ up to $2 \times 10^4 \rightarrow$ sum up to 2×10^8 , safe as `int`, but $10^5 \times 10^5 = 10^{10}$ overflows).
- Use `int` for the remainder key (modulo result is always in `[0, k-1]`).

Why `0L` (not `0`) in the seed

When the map is `Map<Long, Integer>`, the seed **key** must be a `long` literal:

```

Map<Long, Integer> map = new HashMap<>();
map.put(0L, 1);    // ✓ long literal → boxes to Long (matches key type)
map.put(0, 1);    // ✗ compile error: int boxes to Integer, not Long

```

- The `L` suffix makes it a `long` literal so it autoboxes to `Long`, matching the key type (the keys are `long` because `sum` is `long` for the overflow guard above).
- **Subtle trap:** even numerically equal boxes of different types never compare equal — `Long.valueOf(0).equals(Integer.valueOf(0))` is `false`. Keeping every key `long / Long` avoids silent lookup misses.
- If `int` sums can't overflow, use `Map<Integer, Integer>` with plain `map.put(0, 1)` instead — simpler, no `L` needed.

#1 Two Sum

Description: Given an array and a `target`, return the indices of the two numbers that add up to `target`.

Intuition: Same "have I seen the complement before?" idea as prefix-sum, but the key is the raw value, so one pass finds the pair.

Algorithm: HashMap complement lookup. As you scan, for each `nums[i]` check whether its complement `target - nums[i]` was already seen. This is the same "has a value been seen before?" idea as prefix-sum problems, but the key is the raw value (not a running sum).

Variables: `map` = value→index seen so far · `complement` = the partner value `target - nums[i]` we need · `i` = current index
Pseudocode:

```
create empty map of value → index
for i from 0 to n-1:
    complement = target - nums[i]
    if complement already in map:
        return [index of complement, i]
    store nums[i] → i           // after the check, so we never pair with self
return [-1, -1]
```

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>(); // value → index
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];      // ← VARIATION: look up complement, not prefix
            if (map.containsKey(complement))
                return new int[]{map.get(complement), i};
            map.put(nums[i], i);                    // store value → index after the check
        }
        return new int[]{-1, -1};
    }
}
```

Time $O(n)$ | **Space** $O(n)$

- Always normalize Java modulo: `((sum % k) + k) % k` to handle negative inputs.